# Worksheet 5 — Cross-Site Scripting & Client-Side Risks (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 5 Aligned:** OWASP 2025 **A05 Injection** · **CWE-79** (XSS), **CWE-352** (CSRF), **CWE-1004** (cookie without HttpOnly) **Signature game:** ⛳ **XSS Golf** — fire `alert(1)` in the fewest characters possible. Lower payload length = lower score = better. Par for reflected is the `<img>` vector; can you go under par?

> ⚠️ **Ethics note:** Use only the provided `vulnerable_app.py` sandbox and your own Juice Shop container. Stealing real users' cookies or sessions is illegal. All "session theft" steps here target the sandbox cookie `session=abc123` only.

## Part 1 — Student Information

| Name | Student ID | Date | Group |
|---|---|---|---|
| Supichaya Rabiabnaweenurak | 6631503118 | 13/9/2569 | --- |

## Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **reflected**, **stored**, and **DOM-based** XSS by *where* the untrusted data is injected and *when* it executes. Which two does our `vulnerable_app.py` implement, and at which routes?

   > **Answer:** *Reflected* XSS injects the payload into a single response derived straight from the request and executes immediately on that response (here `/hello?name=...`). *Stored* XSS persists the payload server-side and executes later for every visitor who loads the page (here `/comments`). *DOM-based* XSS never touches the server — untrusted data flows from a source like `location.hash` into a dangerous DOM sink in client JS. Our app implements the first two: **reflected at `/hello`** and **stored at `/comments`** (DOM-based is only on the optional Juice Shop target).

2. How does **contextual output encoding** (`markupsafe.escape`) stop `<script>` from executing? Why is HTML-context encoding different from JavaScript- or URL-context encoding?

   > **Answer:** `markupsafe.escape` converts `<`, `>`, `&`, `"`, `'` into HTML entities (`<`, `>`, ...), so `<script>` arrives as literal text the parser renders instead of a tag it executes — the data can no longer become markup. Encoding is context-specific because each context has different metacharacters: inside HTML you escape angle brackets, inside a JavaScript string you must escape quotes/backslashes/newlines, and inside a URL you percent-encode. Applying HTML encoding in a JS or URL context leaves the real dangerous characters for that context unescaped, so the encoder must match where the value lands.

3. Explain how a strict **Content-Security-Policy** (`script-src 'self'`) defeats an *injected* inline script even when encoding is missing.

> **Answer:** `script-src 'self'` tells the browser to run scripts only from the page's own origin and to refuse inline scripts and `on*` event-handler attributes unless explicitly allowlisted (e.g. a nonce/hash). So even if an attacker injects `<script>alert(1)</script>` or `<img onerror=...>` because output encoding failed, the browser blocks execution and reports a CSP violation. It is defense-in-depth: it does not fix the injection, it removes the ability of injected inline code to run.

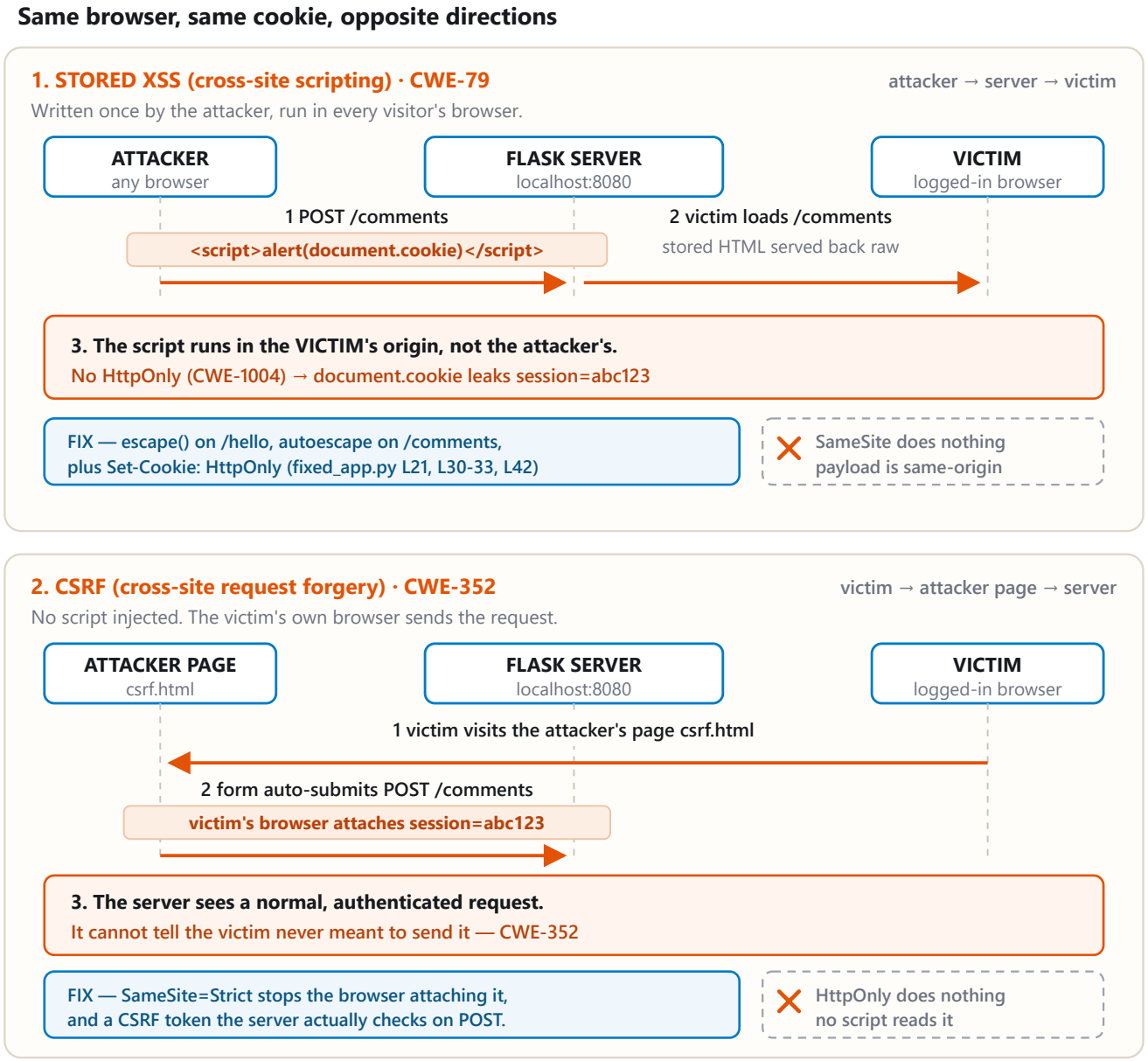
4. What do the cookie flags **HttpOnly**, **SameSite**, and **Secure** each protect against? Map each to a concrete attack (cookie theft via XSS, CSRF, network sniffing).

> **Answer: HttpOnly** hides the cookie from `document.cookie`, so injected JS can't read it — stops **cookie theft via XSS** (Task 3, CWE-1004). **SameSite** (Strict/Lax) tells the browser not to attach the cookie on cross-site requests, so a forged POST from another origin arrives without the session — stops **CSRF** (Task 4). **Secure** sends the cookie only over HTTPS, so it can't be read from plaintext HTTP — stops **network sniffing** / man-in-the-middle capture.

5. Why does **CSRF** (CWE-352) work even without any script injection, and how does `SameSite=Strict` plus the same-origin policy blunt it?

> **Answer:** CSRF needs no injected script because the browser *automatically* attaches the victim's cookies to any request to that site, including a form auto-submitted from an attacker's page; the server can't tell the forged request from a genuine one when it only trusts the cookie. `SameSite=Strict` makes the browser withhold the cookie on cross-site requests, so the forged POST arrives unauthenticated, and the same-origin policy stops the attacker's JS from reading the authenticated response — together they break the silent cross-site abuse. The complete fix also requires a per-request CSRF token the server validates.

## Part 3 — Hands-on Lab (150 min)




- fixed_app.py adds encoding, CSP and HttpOnly — the XSS dies.
- The CSRF still works: /comments checks no token on POST.

**Learning goals:** land reflected + stored XSS, abuse a JS-readable cookie, build a CSRF PoC against the comment board, then prove `fixed_app.py` blocks all of it.

**Prerequisites:** Docker + Docker Compose, a browser with DevTools, a text editor. Working dir: `labs/week05-xss-client-side/`.

## Environment setup

```
cd labs/week05-xss-client-side
docker compose up            # python:3.12-slim + flask, runs vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: xss-lab, port 8080)
```

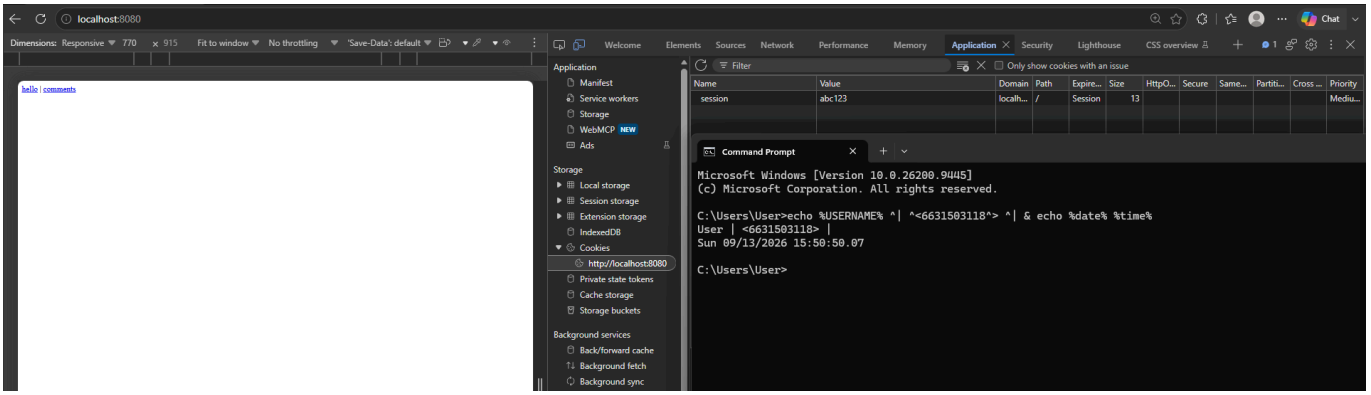
Optional secondary target (for DOM XSS, which our app does not expose):

```
docker run --rm -p 3000:3000 bkimminich/juice-shop       # ->
http://localhost:3000
```

**What to submit per task:** the exact **payload**, a **screenshot** of the alert/effect, and a **2–3 sentence mitigation**.

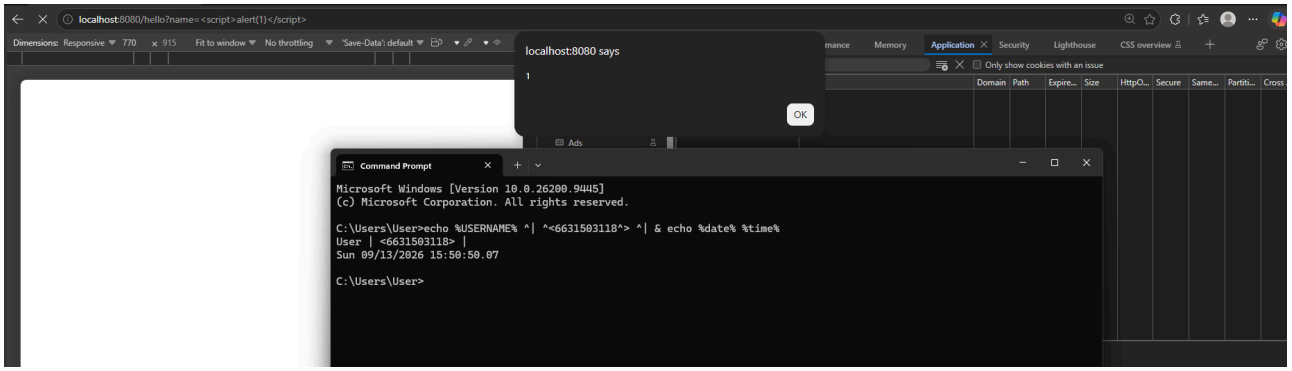
---

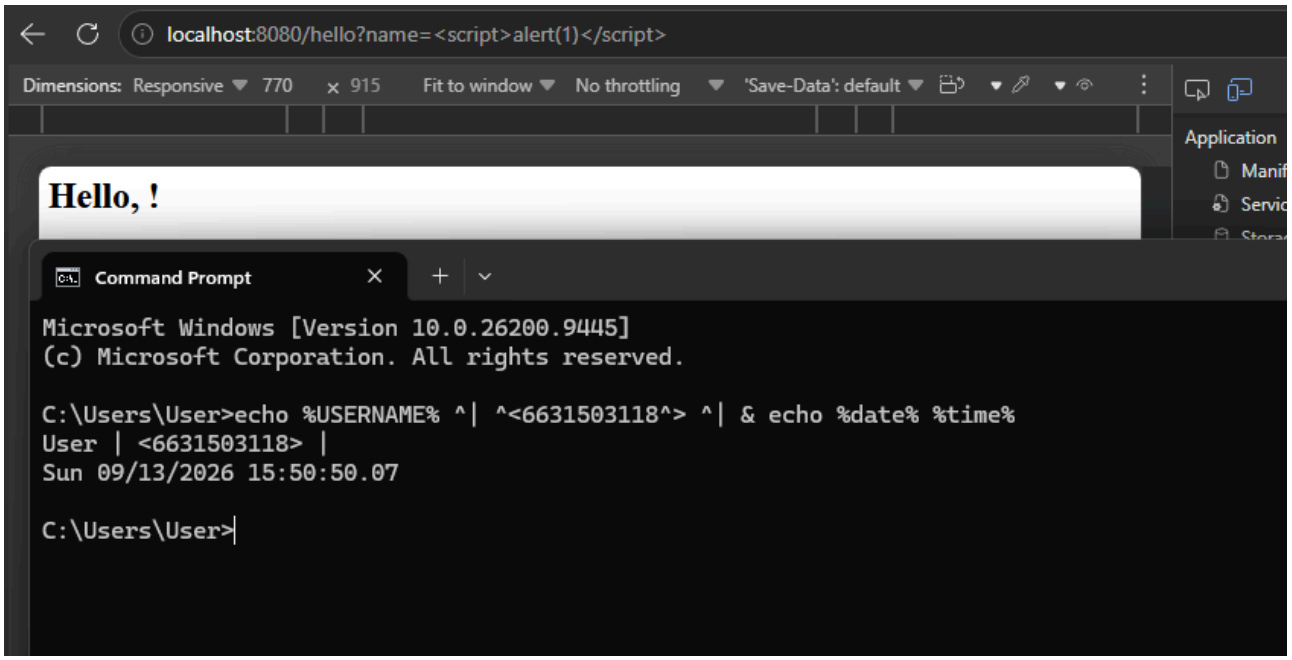
**Task 0 — Onboarding (5 min).** Browse `http://localhost:8080/`. Open DevTools → Application → Cookies and confirm `session=abc123` is set with **no HttpOnly / SameSite**. Screenshot it. *Deliverable: screenshot.*

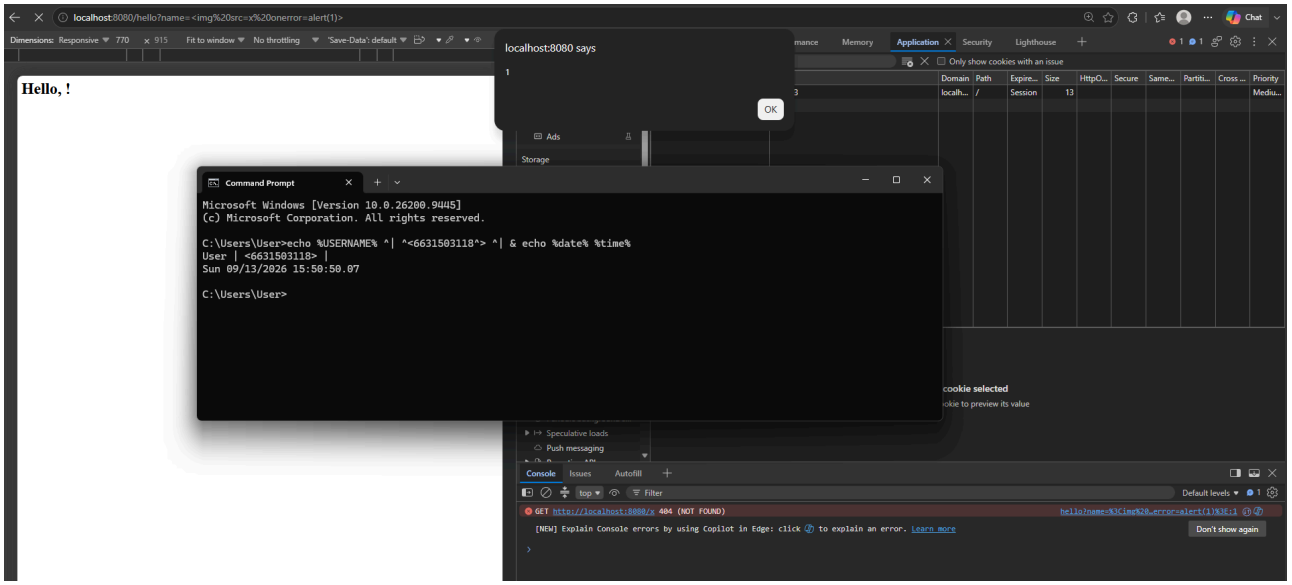


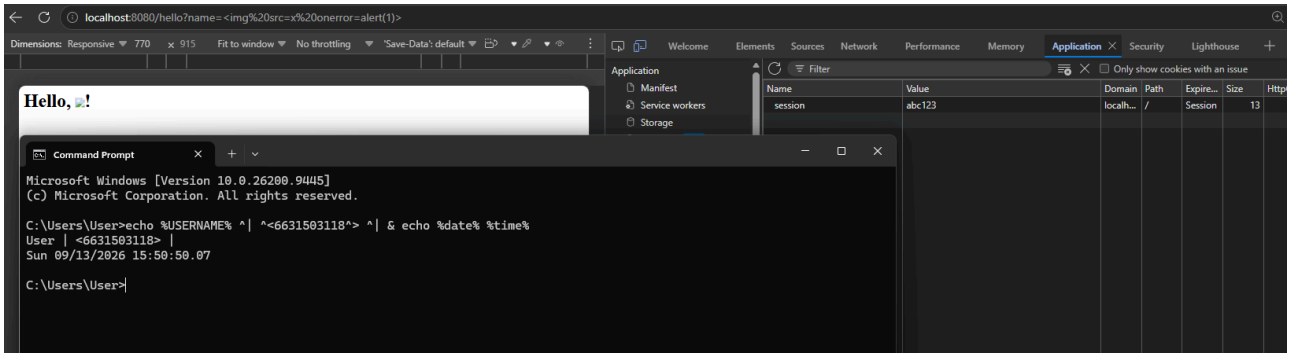

**Task 1 — Reflected XSS + XSS Golf (30 min) ⛳.**

- *Goal:* execute JS via `/hello`, then minimize the payload.
- *Steps:* visit `/hello?name=<script>alert(1)</script>`, then the alternate `/hello?name=<img src=x onerror=alert(1)>` (useful when `<script>` tags specifically are filtered — note it's actually 3 characters longer, not shorter). Record each payload's character count for your golf score.
- *Deliverable:* both payloads + char counts + screenshot of `alert(1)` + your lowest score.





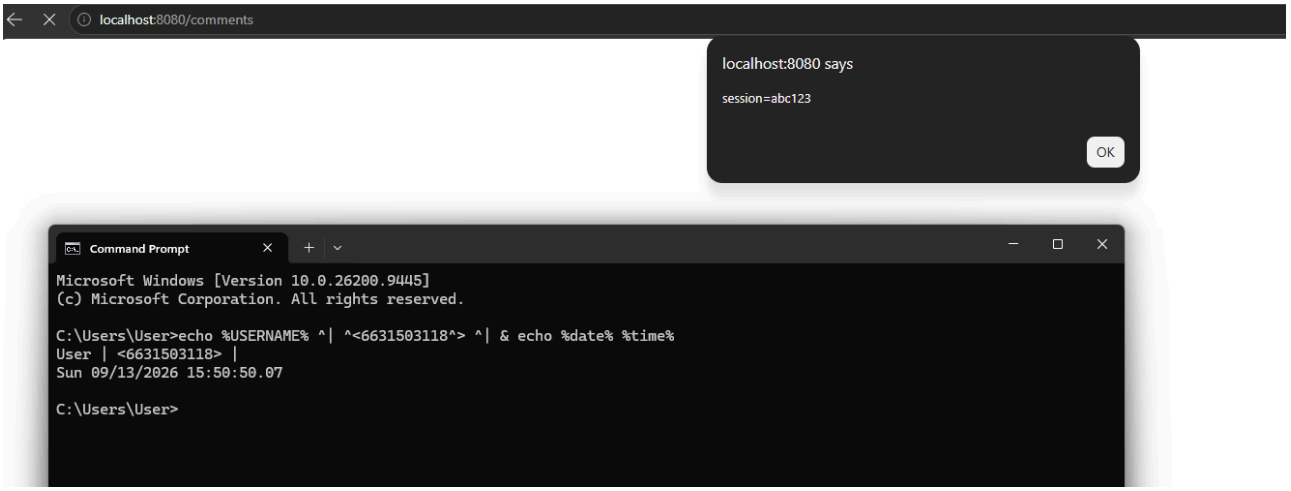

### Task 2 — Stored XSS (30 min) ⛳.

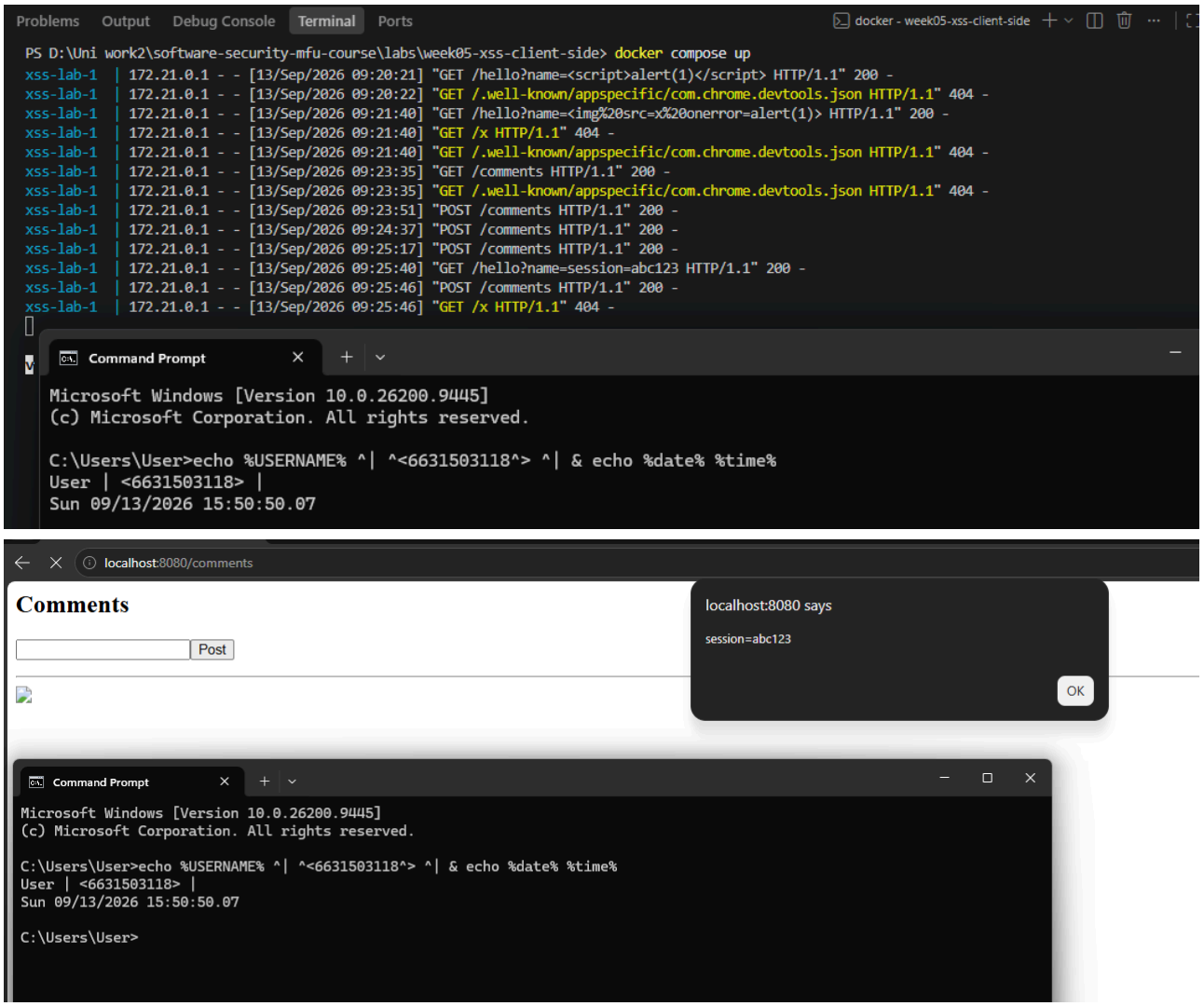
- *Goal:* persist a script that runs for every visitor of `/comments`.
- *Steps:* POST a comment with body `<script>alert(document.cookie)</script>` (use the form or `curl -d 'body=...'`). Reload `/comments` and watch the cookie pop.
- *Deliverable:* payload + screenshot of the alert showing `session=abc123` + why stored XSS is more dangerous than reflected.




**Task 3 — Cookie theft via XSS (25 min).**

- *Goal:* show the cookie is readable by injected JS because **HttpOnly is missing** (CWE-1004).
- *Steps:* store `<script>new Image().src='http://localhost:8080/hello?name='+document.cookie</script>` (a beacon), or simply `<img src=x onerror=alert(document.cookie)>`. Observe the cookie value being exfiltrated/displayed.
- *Deliverable:* payload + screenshot + 2–3 sentences on how HttpOnly would have stopped this.









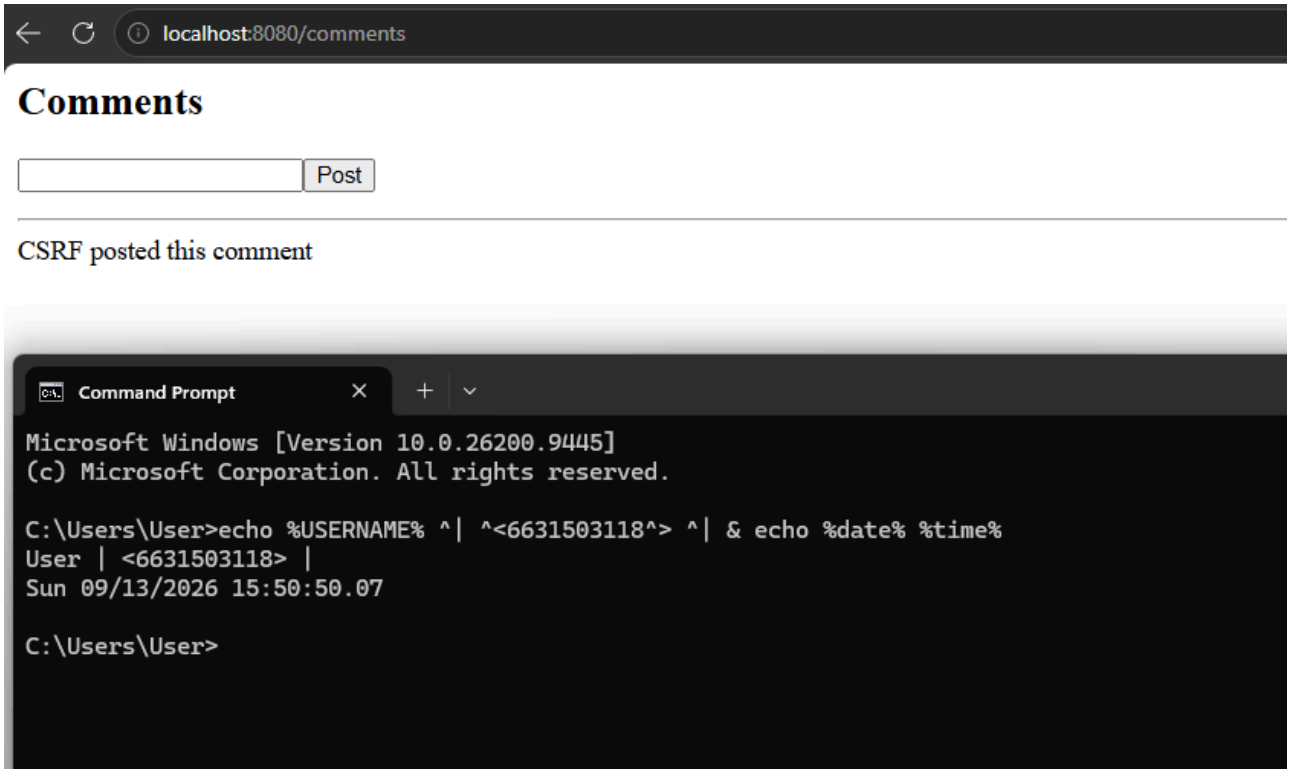
**Task 4 — CSRF PoC (30 min).**

- *Goal:* make a third-party page force a state-changing POST to `/comments`.
- *Steps:* create a local `csrf.html` with an auto-submitting form targeting the board (no token exists, cookie has no SameSite, so the browser attaches `session` cross-site):

```
<body onload="document.forms[0].submit()">
  <form action="http://localhost:8080/comments" method="POST">
    <input name="body" value="CSRF posted this comment">
  </form>
</body>
```

Open the file and confirm the comment appears on `/comments`.

- *Deliverable:* the HTML + screenshot of the forged comment + why `SameSite=Strict` blocks it.




```
xss-context
```

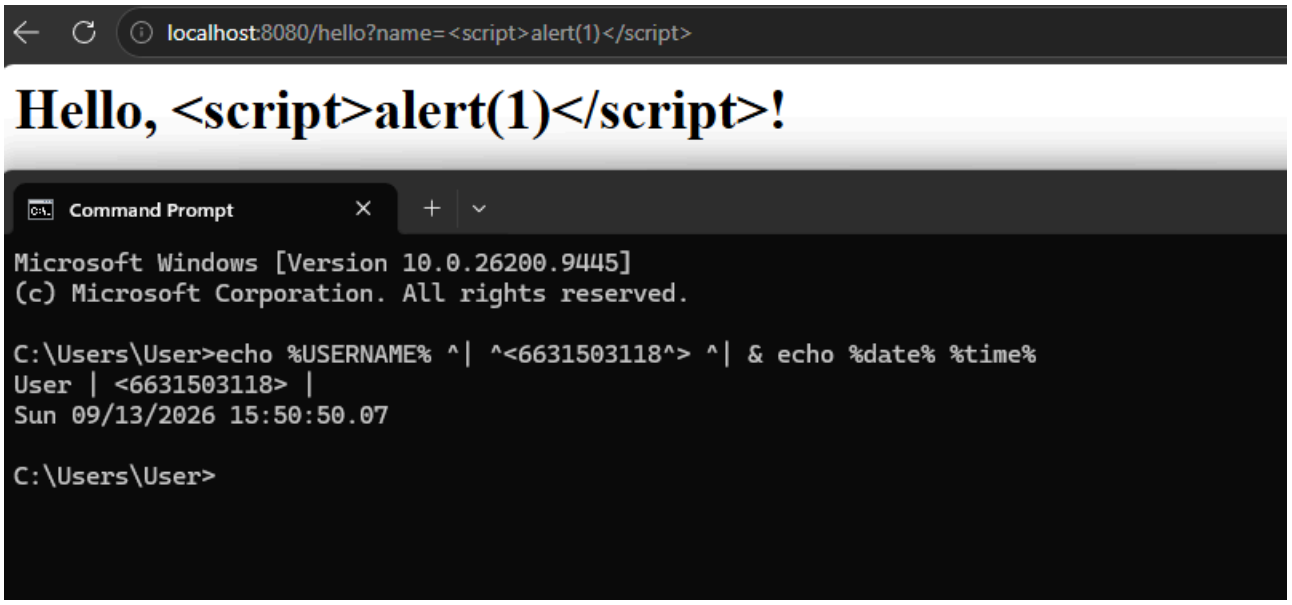
**Task 5 — Defend / fix it (30 min) 🛡️.**

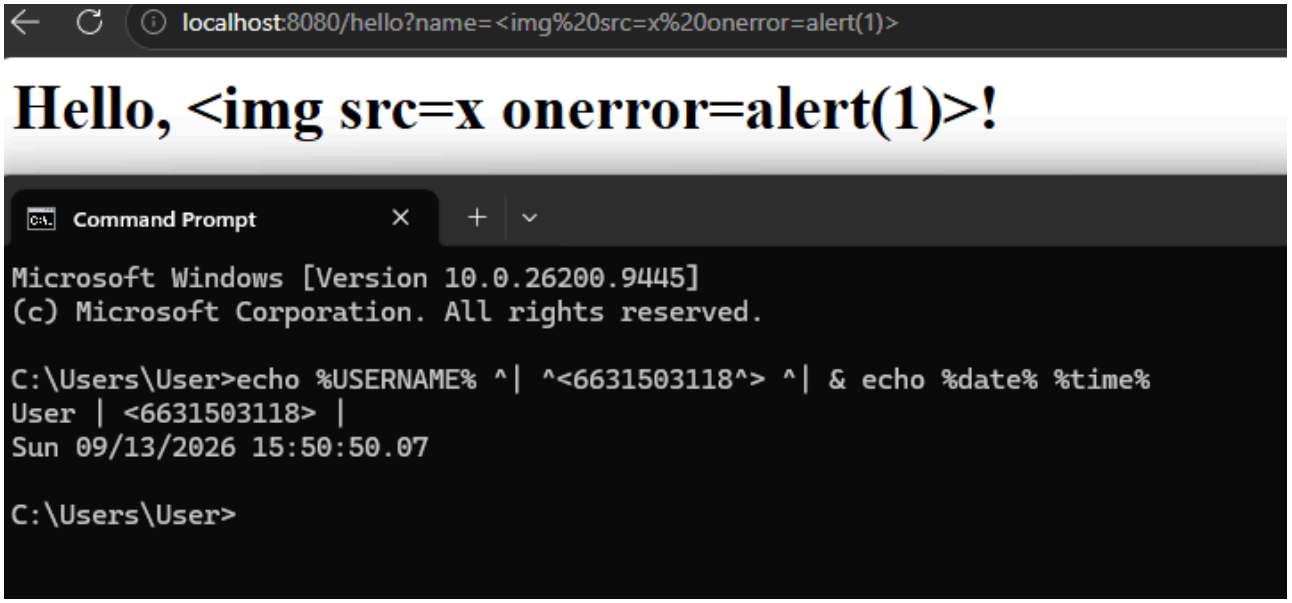
- *Goal:* prove `fixed_app.py` blocks Tasks 1–3, then show that Task 4's CSRF PoC still gets through and explain why.
- *Steps:* stop the vulnerable container (`Ctrl-C`), then:

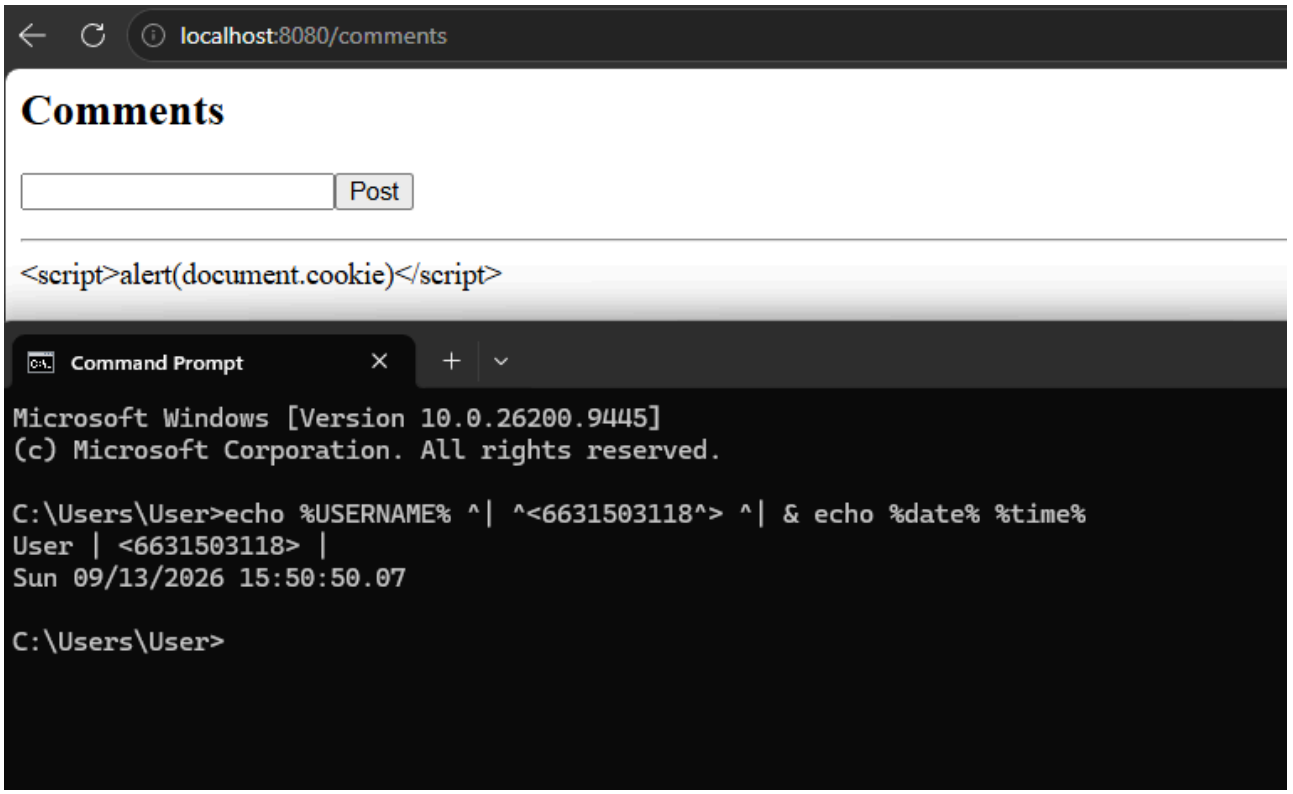
  ```
  docker compose run --rm --service-ports xss-lab bash -c "pip install --no-cache-dir flask && python fixed_app.py"
  ```

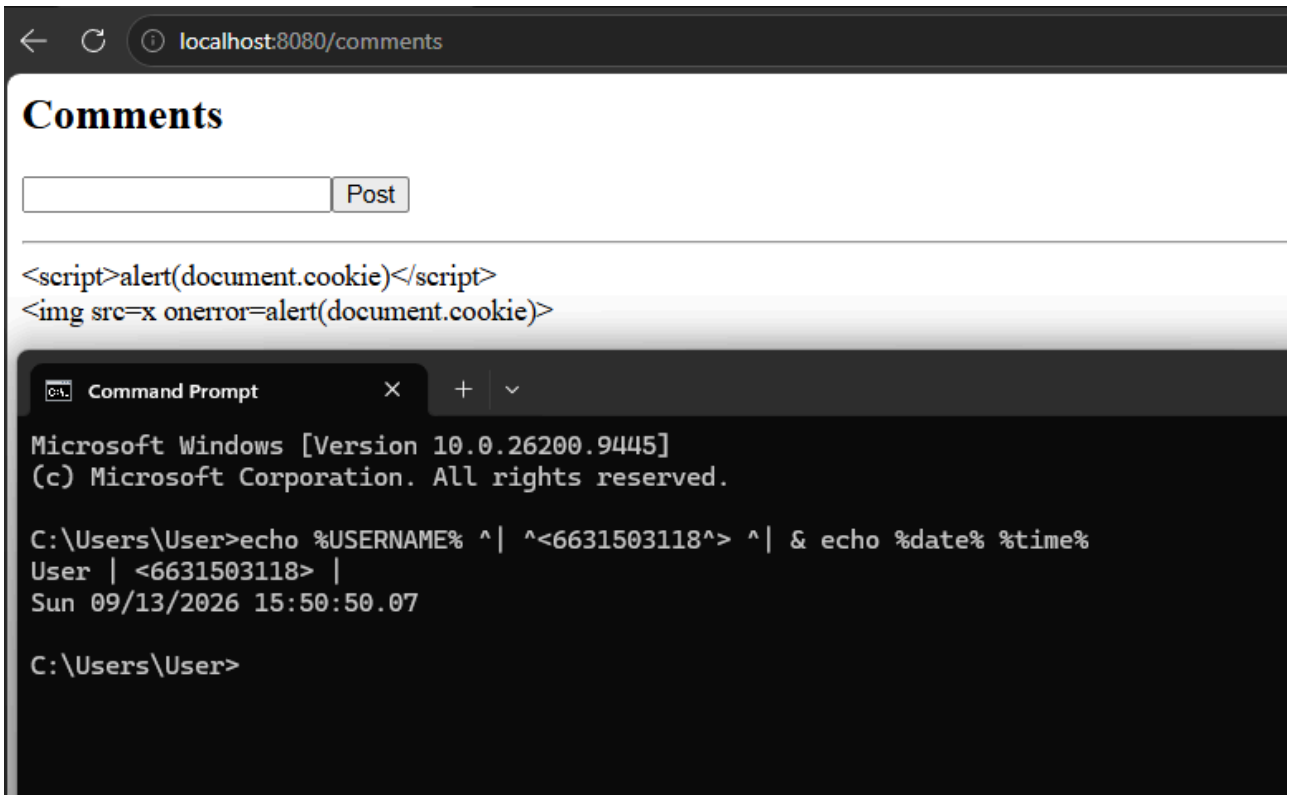
  Re-fire each payload. Expected: `/hello` renders the script **as text** (escape, L21), stored comments render literally (Jinja autoescape, L30–33), a strict CSP header is now present as defense-in-depth (`Content-Security-Policy: script-src 'self'`, L12 — check DevTools → Network → Response Headers; escaping already neutralizes these payloads, so no CSP *violation* fires in the console), and the cookie now has `HttpOnly; SameSite=Strict; Secure` (L42). Then re-run Task 4's `csrf.html` PoC against `fixed_app.py`: it **still posts the forged comment** — `/comments` (L25–28) never checks the `session` cookie or a CSRF token before accepting a POST, so hardening the cookie only stops the browser from *attaching* it cross-site; it doesn't stop the request itself from being processed.
- *Deliverable:* screenshots of escaped output + the CSP response header + the hardened cookie flags + the still-successful Task 4 forgery against `fixed_app.py`, with 2–3 sentences on why cookie hardening alone doesn't close CSRF here (no server-side check tied to the cookie, and no CSRF token).

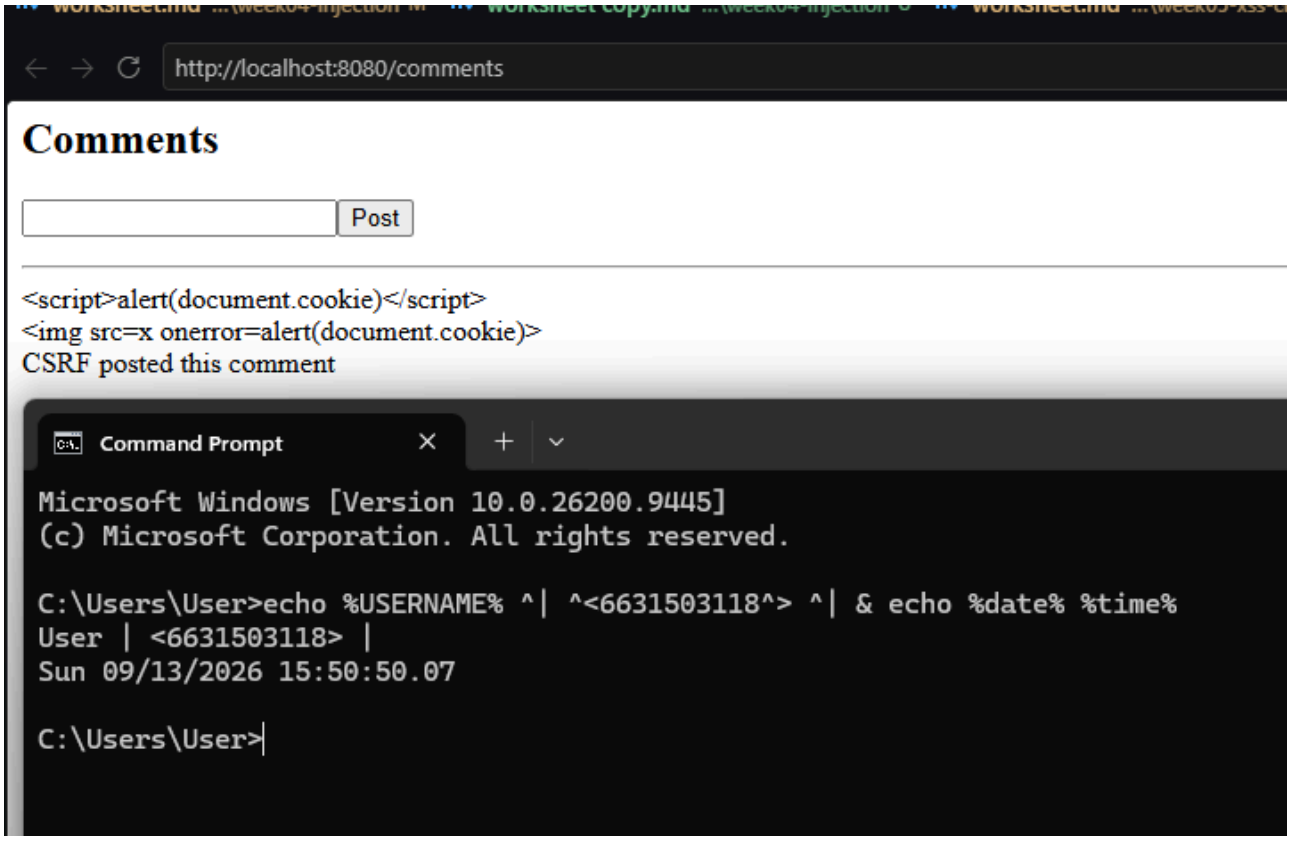

localhost:8080/hello
Hello, world!
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>


localhost:8080/hello?name=<script>alert(1)</script>
Hello, <script>alert(1)</script>!
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>


localhost:8080/hello?name=<img%20src=x%20onerror=alert(1)>
Hello, <img src=x onerror=alert(1)>!
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>


localhost:8080/comments
Comments
Post
<script>alert(document.cookie)</script>
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>


localhost:8080/comments
Comments
Post
<script>alert(document.cookie)</script>
<img src=x onerror=alert(document.cookie)>
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>




## Part 4 — Reflection

1. **CWE/OWASP mapping:** map your reflected/stored XSS to **CWE-79** and your CSRF PoC to **CWE-352**, both under OWASP 2025 **A05 Injection** (CSRF historically A01/A05).

> **Answer:** Reflected XSS (`/hello`) and stored XSS (`/comments`) are both **CWE-79 (Improper Neutralization of Input During Web Page Generation)**; the cookie-theft beacon is the same CWE-79 abused to read a cookie left exposed by **CWE-1004 (cookie without HttpOnly)**. The CSRF PoC is **CWE-352 (Cross-Site Request Forgery)**. All map to OWASP 2025 **A05 Injection** (CSRF has historically also sat under A01 Broken Access Control / earlier A05).

2. **Real breach:** the **2018 British Airways breach** (~380k payment records) used malicious JavaScript (Magecart) injected into the site to skim card data — a client-side script-injection failure. In 3–4 sentences relate it to this lab's XSS and CSP lessons.

> **Answer:** In the 2018 British Airways breach, Magecart attackers injected malicious JavaScript into the site so that ~380k customers' card details were skimmed as they were typed and exfiltrated to an attacker server — a client-side script-injection failure, exactly the class this lab's XSS tasks demonstrate. Just as our stored-XSS payload runs in every visitor's browser in the site's own origin, the BA script ran with full trust on the payment page. A strict Content-Security-Policy (`script-src 'self'`, no unlisted external/inline scripts) is precisely the control that would have blocked the injected skimmer from loading or exfiltrating, which is why CSP matters as defense-in-depth on top of output encoding.

3. **Best mitigation:** between output encoding, a strict CSP, and HttpOnly+SameSite cookies, which gives the broadest defense-in-depth, and why is "encoding alone" still risky?

> **Answer:** A **strict CSP** gives the broadest defense-in-depth because it neutralizes *any* injected inline/external script regardless of which sink or encoding gap let it in, and it protects every page uniformly. Encoding alone is still risky because it must be applied correctly in *every* output context (HTML, attribute, JS, URL) on *every* sink — a single missed or wrong-context escape re-opens XSS — and it does nothing once a script does execute. The strongest posture layers all three: encode output (prevent injection), CSP (contain execution), and HttpOnly+SameSite (limit the damage if a script still runs).

## Grading rubric (100)

| Criterion | Points |
|---|---|
| Part 2 — Lecture questions (conceptual accuracy) | 20 |
| Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4) | 40 |
| Part 3 — Defense (Task 5: fixes proven, lines cited) | 25 |
| Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation) | 15 |
| **Total** | **100** |

---

## Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*

  1. What did you do, and **why did the vulnerability work**?

> *Draft to rewrite in your own words:* I injected script via `/hello` (reflected) and `/comments` (stored), read the `session=abc123` cookie because `HttpOnly` was missing, and forced a cross-site POST with `csrf.html`. It worked because the app concatenates untrusted input straight into HTML without encoding, so the data became executable markup, and because the cookie had no `HttpOnly`/`SameSite` and `/comments` had no CSRF token.

  2. **Why does your fix actually stop it** — and what could still break it?

> *Draft to rewrite in your own words:* `fixed_app.py` escapes output and relies on Jinja autoescape so the payload renders as text, adds `Content-Security-Policy: script-src 'self'` so injected inline scripts can't run, and sets `HttpOnly; SameSite=Strict; Secure` on the cookie. It could still break the CSRF case: hardening the cookie only stops the browser *attaching* it cross-

> site — `/comments` still accepts any POST because there is no server-side CSRF token, so the forged comment still lands.

---

## 🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

> *Example AI answer (replace with your own run):*
>
> ```
> # AI's "fix" for stored XSS on /comments
> body = request.form.get("body", "").replace("<script>", "").replace("
> </script>", "")
> COMMENTS.append(body)
> ```
>
> *Critique (quote the lines):* The `.replace("<script>", "")` line is a blocklist and is trivially bypassed — `<img src=x onerror=alert(1)>` contains no `<script>` tag, and `<SCRIPT>` / `<scr<script>ipt>` also defeat it. It also encodes nothing on output, so the stored value is still rendered as raw HTML. This is the classic sanitize-by-substring mistake.
>
> *Correct, verified version:*
>
> ```
> from markupsafe import escape
> # store raw, escape on output (or let Jinja autoescape render it):
> COMMENTS.append(request.form.get("body", ""))
> ...
> body += "".join(" " + str(escape(c)) + " " for c
> in COMMENTS)
> ```
>
> *Why the AI's was insufficient:* Stripping specific tags is a deny-list that misses the many other XSS vectors (event handlers, other tags, mixed case, nested tags), whereas contextual output encoding converts every dangerous character to an entity so the browser can never parse the value as markup. Verified by re-firing `<img src=x onerror=alert(1)>`: it now renders as literal text and no alert fires.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

---

## 🧠 Comprehension & Prompt (required)

**A. Explain in Plain English (EiPE).** In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

> *Draft to rewrite in your own words:* `/hello` echoes the `name` query value straight into the HTML response and `/comments` stores each comment and later concatenates it into the page as raw HTML, with no encoding in either case. It's exploitable because the browser can't tell the app's intended markup from attacker text, so input like `<script>...</script>` or `<img onerror=...>` is parsed and executed as code in the victim's origin — letting it read the non-HttpOnly cookie or act as the user.

**B. Prompt Problem.** Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

> *Final prompt:* "This Flask comment board is vulnerable to stored XSS because it concatenates `request.form['body']` into the HTML response unescaped. Fix it using contextual HTML output encoding (`markupsafe.escape`) or Jinja autoescaping — not a blocklist of tags — and also add a `Content-Security-Policy: script-src 'self'` response header as defense-in-depth. Show the full fixed route. [paste /comments code]"
>
> *Verified result:* After applying the fix, re-posting `<script>alert(document.cookie)</script>` and `<img src=x onerror=alert(1)>` shows both rendered as literal text with no alert, and the response carries the `script-src 'self'` header — exploit fails. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*